04. Divide-and-Conquer

Hyunjong Choi, Ph.D.
Assistant Professor of Computer Science
San Diego State University

Outline

- More algorithms based on ***divide-and-conquer***
  - Maximum-subarray problem (Textbook 3rd edition 68 – 74)
  - Matrix multiplication problem (straightforward method vs. Strassen's algorithm)

- Solving recurrence equations
  - Substitution method for solving recurrence
  - Recursion-tree method
  - Master method (master theorem)

Recall Merge sort

❑ Merge sort is a *divide-and-conquer* algorithm. It includes three basic steps:

- **Divide** the problem into a number of subproblems that are smaller instances of the same problem
- **Conquer** the subproblems by solving them recursively

Recursive case: when a subproblems are large enough to solve recursively, e.g., A.length > 2

Base case: when a subproblems become small enough that we no longer recurse, e.g., A.length = 1

- **Combine** the solutions to the subproblems into the solution for the original problem

Recurrence

- An equation or inequality that describes a function in terms of its value on smaller inputs
- For divide-and-conquer algorithms, use recurrence to describe the running time

e.g., running time of MERGE-SORT: $T(n) = \begin{cases} \Theta(1), & \text{if } n = 1 \\ 2T(n/2) + \Theta(n), & \text{if } n > 1 \end{cases}$

Solution was $\boldsymbol{T(n) = \Theta(nlgn)}$

- Other forms of recurrences
  - Unequal sizes, e.g., $T(n) = T\left(\frac{2n}{3}\right) + T\left(\frac{n}{3}\right) + \Theta(n)$
  - Not a constant fraction, e.g., $T(n) = T(n-1) + \Theta(1)$

Solving recurrences

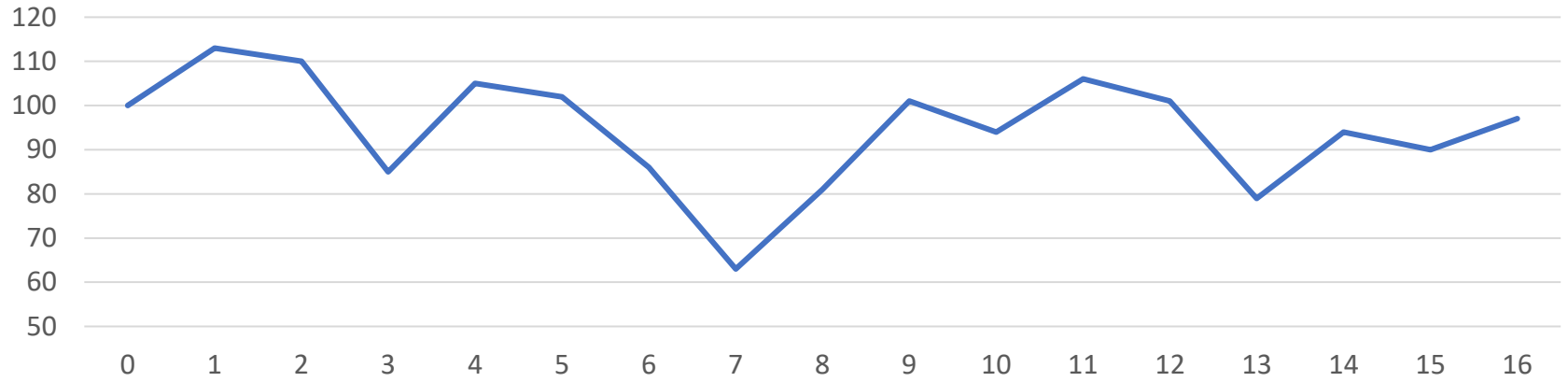
❑ How to solve $T(n)$, i.e., obtain its Θ or O bounds?

- **Substitution method**:
  - ✓ Guess the form of a bound and then use mathematical induction to prove our guess correct.
  - ✓ Robust method, but requires you to make a good guess and to produce an inductive proof.
- **Recursion-tree method**:
  - ✓ Models the recurrence as a tree whose *nodes represent the costs*
  - ✓ Determine the costs at each level and add them up
  - ✓ Even if you don't use this method to prove a bound formally, it can be helpful in guessing the form of the bound for use in the substitution method.
- **Master method**: *General* solutions for recurrences of the form:
  - ✓ $T(n) = aT\left(\frac{n}{b}\right) + f(n)$, where $a \geq 1$, and $b > 1$

Maximum subarray problem

- If you have an opportunity to invest,
  - **Goal**: to maximize the profit, "***buy low, sell high***"
  - Buy one unit at one time, and then sell it at a later time.

Day	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Price	100	113	110	85	105	102	86	63	81	101	94	106	101	79	94	90	97

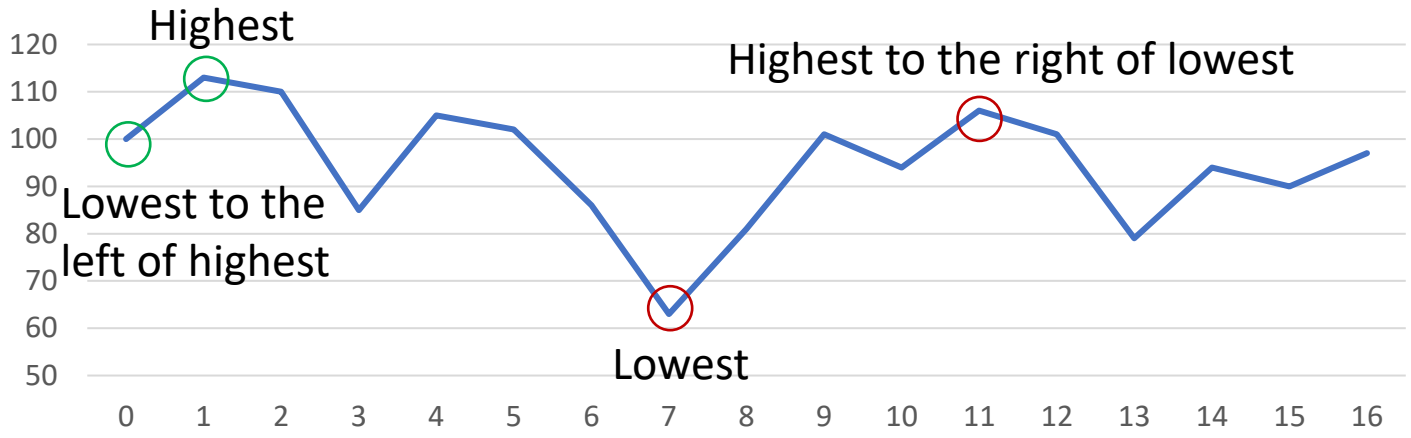


< Price by day >

A straightforward approach

- Buying at the lowest and selling at the highest?
  - Find the lowest and highest prices
    - ✓ **Method 1**. From the highest price scan left to find the lowest price
    - ✓ **Method 2**. From the lowest price scan right to find the highest price

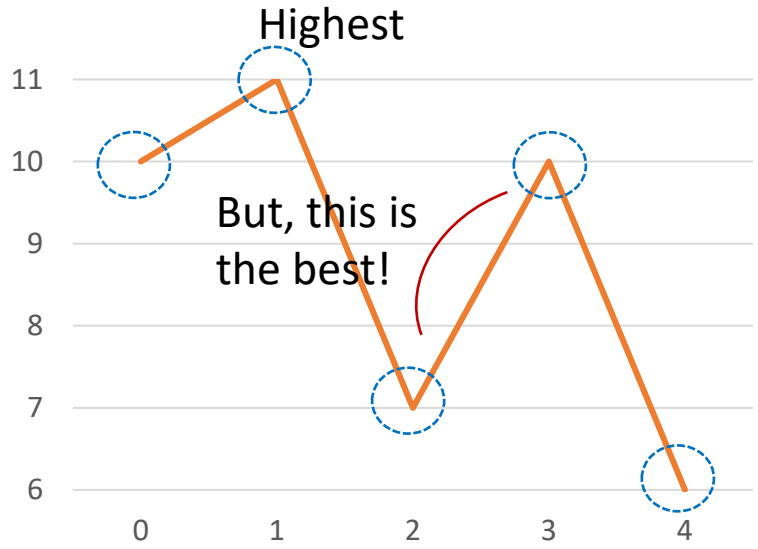
    Then, take the pair with the greater difference



Best strategy: buy at Day 7 and sell at Day 11

A straightforward approach (Cont.)

❑ Let's take a look another case,



- According to this strategy, should buy at Day 0 and sell at Day 1
- However, the best strategy is to Buy at Day 2 and sell at Day 3
- Neither Day 2 nor Day 3 has the global minimum or maximum price!

➡ **Therefore, a straightforward algorithm does not always work!**

A brute-force solution

❑ Try ***every possible pair*** of buy and sell dates

```
BRUTE-FORCE-FIND-MAX(A, low, high)
1.  max-profit = 0
2.  max-left = Nil, max-right = Nil
3.  for i = low to high – 1
4.      for j = i + 1 to high
5.          profit = A[j] – A[i]
6.          if profit > max-profit
7.              max-left = i
8.              max-right = j
9.              max-profit = profit
10. return (max-left, max-right, max-profit)
```

Two nested for loops

- If $A.length = n$, the statement of the inner loop is executed this many times:

$$(n-1) + (n-2) + \cdots + 1$$
$$= \frac{n(n-1)}{2} = \Theta(n^2)$$

❑ Can we do better?

A transformation of the problem

- Rephrase the problem: Find a sequence of days over which the **net change** from the first day to the last is **maximum**.
- Consider the daily change: the change on day i is the $Price[i] - Price[i-1]$

Day	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Price	100	113	110	85	105	102	86	63	81	101	94	106	101	79	94	90	97
A ← Change		13	-3	-25	20	-3	-16	-23	18	20	-7	12	-5	-22	15	-4	7

Maximum subarray: $sum(A[8:11]) = 43$

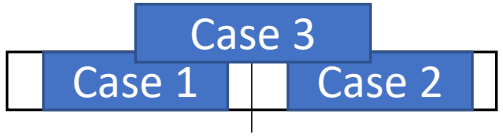
- Find the contiguous subarray of A whose values have the largest sum.
- We call this contiguous subarray the **maximum subarray**.

How does the transformation help?

- It does not help so far.
- Still need to check $\binom{n-1}{2} = \Theta(n^2)$ subarrays.
- We need a more efficient solution: Divide-and-conquer

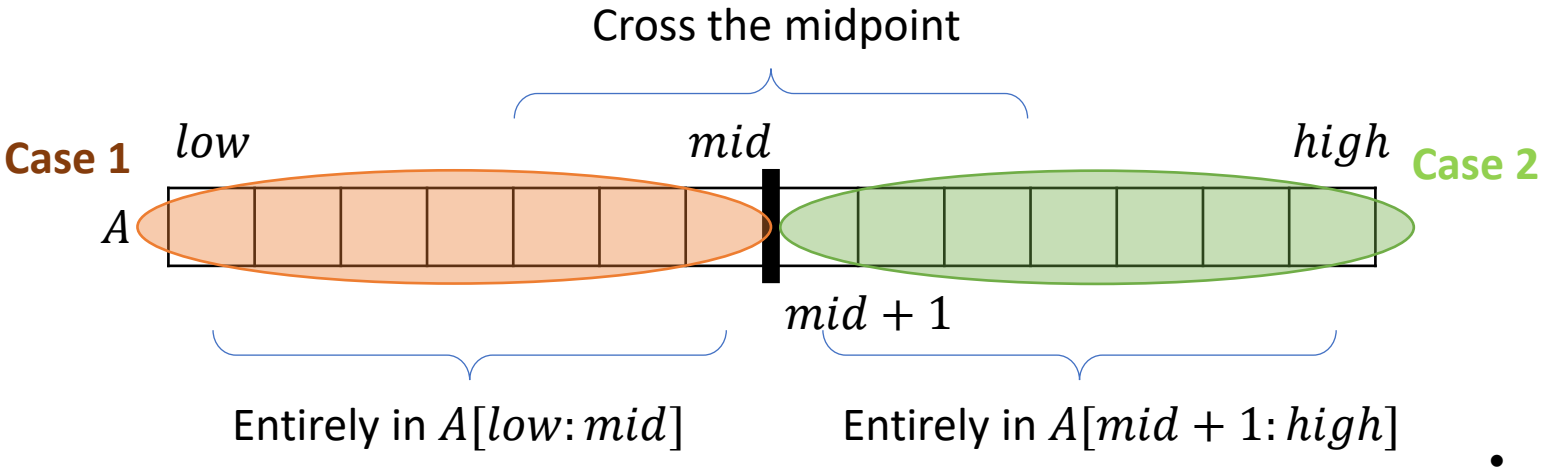
A solution using divide and conquer

- In order to find a maximum subarray of $A[low: high]$
  - **Divide :** split A into two subarrays: $A[low: mid]$ and $A[mid + 1: high]$, where mid is the midpoint
  - Then *any* subarray $A[i: j]$ must be exactly one of the three cases:
    - ✓ Case 1: entirely in the **left** subarray $A[low: mid]$, i.e., $low \leq i \leq j \leq mid$
    - ✓ Case 2: entirely in the **right** subarray $A[mid + 1: high]$, i.e., $mid + 1 \leq i \leq j \leq high$
    - ✓ Case 3: **crossing** the midpoint, i.e., $low \leq i \leq mid < j \leq high$



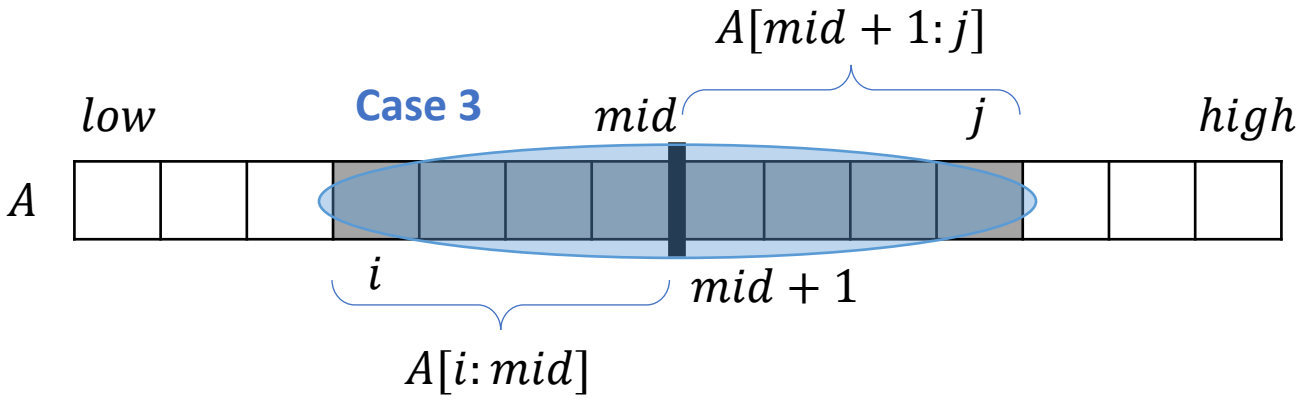
  - **Conquer :** Solve all cases
    - ✓ Solve the maximum subarrays of $A[low: mid]$ **(Case 1)** and $A[mid + 1: high]$ **(Case 2)** *recursively*, because they are smaller instances of the original problems.
    - ✓ Solve a maximum subarray that *crosses* the midpoint **(Case 3)**
  - **Combine :** take a subarray with the largest sum of the **three items**.

A solution using divide-and-conquer



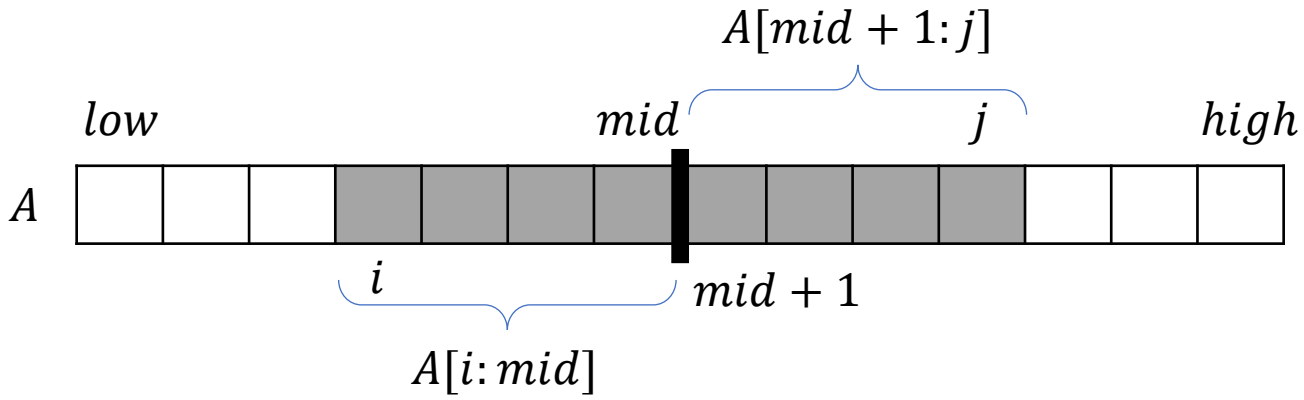
- **Divide-and-conquer**: Find the maximum subarray with the largest sum among: $A[low: mid]$, $A[mid + 1: high]$, and $A[i: j]$

- Any subarray $A[i: j]$ that crosses the mid point



Case 3: subarray crossing the midpoint

- This is **not** a smaller instance of the original problem.
- Any subarray crossing the midpoint itself consists of two parts:
  - Left half $A[i: mid]$, right half $A[mid + 1: j]$



Case 3: subarray crossing the midpoint

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
1.  left-sum = −∞
2.  sum = 0
3.  for i = mid downto low
4.      sum = sum + A[i]
5.      if sum > left-sum
6.          left-sum = sum
7.          max-left = i
8.  right-sum = −∞
9.  sum = 0
10. for j = mid + 1 to high
11.     sum = sum + A[j]
12.     if sum > right-sum
13.         right-sum = sum
14.         max-right = j
15. return (max-left, max-right, left-sum + right-sum)
```

Hold the greatest sum found so far for the left half

Work down from *mid* to *low* to examine every subarray in the left half

Whenever a greater sum is found, update *left-sum* and record the index to *max-left*

Similar operations for the right half

Return the indices *max-left* and *max-right* that indicate a maximum subarray, and its sum of values

Iterate over $(mid - low + 1)$ entries on the left

Iterate over $(high - mid)$ entries on the right

Total # of iterations:

$$(mid - low + 1) + (high - mid) = high - low + 1 = A[low: high].length = n$$

It takes $\Theta(n)$ time

Implement divide-and-conquer

❑ FIND-MAXIMUM-SUBARRAY by divide-and-conquer approach

```
FIND-MAXIMUM-SUBARRAY(A, low, high)
1.   if high == low
2.         return (low, high, A[low])
3.   else mid = ⌊(low + high)/2⌋
4.         (left-low, left-high, left-sum) =
5.               FIND-MAXIMUM-SUBARRAY(A, low, mid)
6.         (right-low, right-high, right-sum) =
7.               FIND-MAXIMUM-SUBARRAY(A, mid + 1, high)
8.         (cross-low, cross-high, cross-sum) =
9.               FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
10.        if left-sum ≥ right-sum and left-sum ≥ cross-sum
11.              return (left-low, left-high, left-sum)
12.        elseif right-sum ≥ left-sum and right-sum ≥ cross-sum
13.              return (right-low, right-high, right-sum)
14.        else return (cross-low, cross-high, cross-sum)
```

Handle the base case: a subarray of one element → return itself

Divide

Conquer the left half by calling FIND-MAXIMUM-SUBARRAY recursively

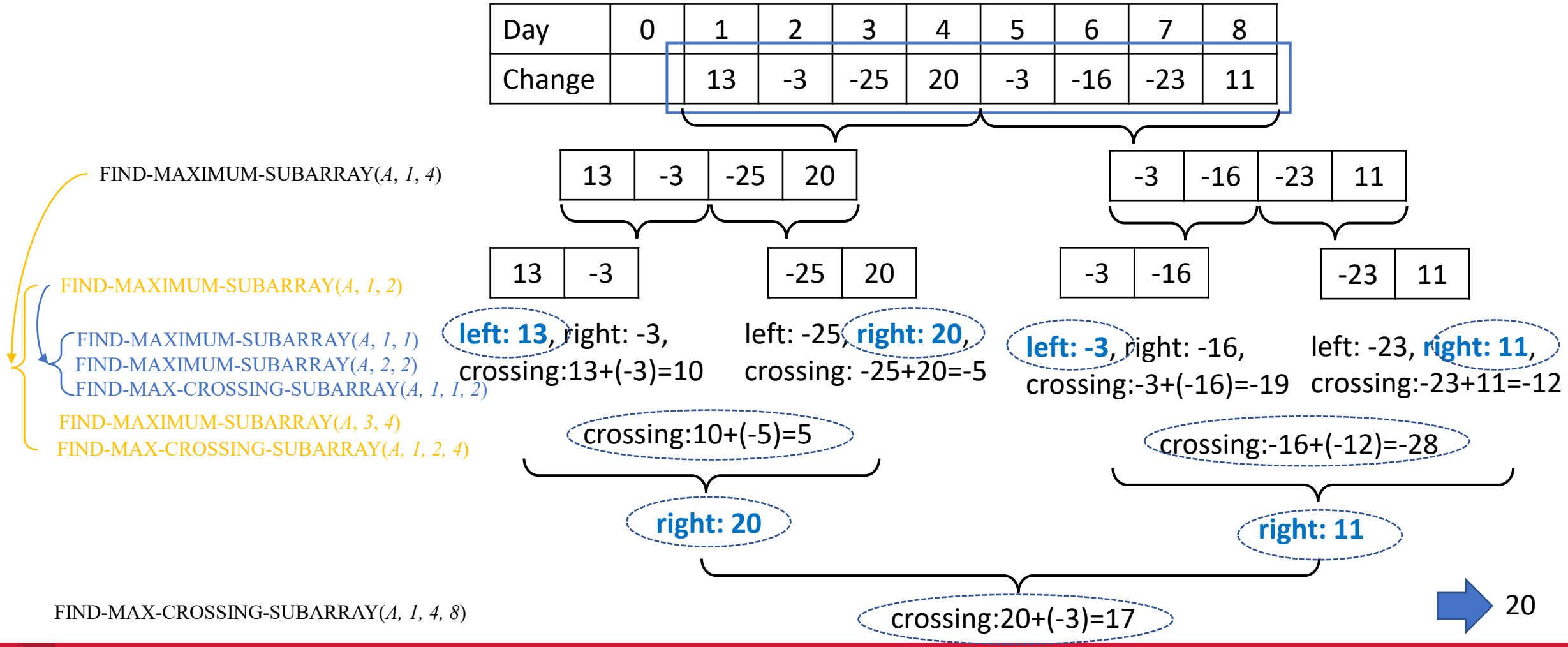
Conquer the right half by calling FIND-MAXIMUM-SUBARRAY recursively

Find a maximum subarray that crosses the midpoint.

Combine: Test which one of *left-sum*, *right-sum*, and *cross-sum* is the largest, and return the corresponding indices and sum value

Example 1.

- Run FIND-MAXIMUM-SUBARRAY(*A*, *1*, *8*)



Example 2.

- Run FIND-MAXIMUM-SUBARRAY(A, 1, 16)

A

Day	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Change		13	-3	-25	20	-3	-16	-23	18	20	-7	12	-5	-22	15	-4	7

$mid = \lfloor (1 + 16)/2 \rfloor = 8$

FIND-MAXIMUM-SUBARRAY(A, 1, 8)

FIND-MAXIMUM-SUBARRAY(A, 9, 16)

FIND-MAX-CROSSING-SUBARRAY(A, 1, 8, 16) → cross-low = 8, cross-high = 11, cross-sum = 43

Example (cont.)

- Recursive call: FIND-MAXIMUM-SUBARRAY(A, 1, 8)

A

Day	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Change		13	-3	-25	20	-3	-16	-23	18	20	-7	12	-5	-22	15	-4	7

$mid = \lfloor (1 + 8)/2 \rfloor = 4$

FIND-MAXIMUM-SUBARRAY(A, 1, 4)

FIND-MAXIMUM-SUBARRAY(A, 5, 8)

FIND-MAX-CROSSING-SUBARRAY(A, 1, 4, 8) → cross-low = 4, cross-high = 5, cross-sum = 17

Example (cont.)

- Recursive call: FIND-MAXIMUM-SUBARRAY(A, 1, 4)

	Day	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
A	Change		13	-3	-25	20	-3	-16	-23	18	20	-7	12	-5	-22	15	-4	7

$mid = \lfloor (1 + 4)/2 \rfloor = 2$

FIND-MAXIMUM-SUBARRAY(A, 1, 2) → left-low = 1, left-high = 1, left-sum = 13

FIND-MAXIMUM-SUBARRAY(A, 3, 4) → right-low = 4, right-high = 4, right-sum = 20

FIND-MAX-CROSSING-SUBARRAY(A, 1, 2, 4) → cross-low = 1, cross-high = 4, cross-sum = 5

Analyze the running time

```
FIND-MAXIMUM-SUBARRAY(A, low, high)
1.   if high == low
2.        return (low, high, A[low])
3.   else mid = ⌊(low + high)/2⌋
4.        (left-low, left-high, left-sum) =
5.             FIND-MAXIMUM-SUBARRAY(A, low, mid)
6.        (right-low, right-high, right-sum) =
7.             FIND-MAXIMUM-SUBARRAY(A, mid + 1, high)
8.        (cross-low, cross-high, cross-sum) =
9.             FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
10.       if left-sum ≥ right-sum and left-sum ≥ cross-sum
11.            return (left-low, left-high, left-sum)
12.       elseif right-sum ≥ left-sum and right-sum ≥ cross-sum
13.            return (right-low, right-high, right-sum)
14.       else return (cross-low, cross-high, cross-sum)
```

- When $n = 1$, line 2 is run, which takes constant time, i.e., $\Theta(1)$. Thus, $T(1) = \Theta(1)$.
- When $n > 1$, line 3 is first run, which also takes $\Theta(1)$ time.
- Lines 4 and 5 solve the subproblem on a subarray of $n/2$ elements, which takes $T(n/2)$ time.
- Similarly, line 6 and 7 also take $T(n/2)$ time.
- We already know that FIND-MAX-CROSSING-SUBARRAY on a subarray of n elements takes $\Theta(n)$ time.
- Combine part takes $\Theta(1)$ time.

Altogether, when $n > 1$

$$T(n) = \Theta(1) + 2T(n/2) + \Theta(n) + \Theta(1)$$
$$= 2T(n/2) + \Theta(n)$$

Complete recurrence

- Combine the base case ($n = 1$) and the recurrent case ($n > 1$)

$$T(n) = \begin{cases} \Theta(1), & \text{if } n = 1 \\ 2T(n/2) + \Theta(n), & \text{if } n > 1 \end{cases}$$

- Looks familiar? **same form** as merge sort

- Therefore, $\boldsymbol{T(n) = \Theta(n \cdot \lg n)}$
- Remember how we solved it using the recursion tree plot?

Example

Problem

- Find the maximum sub-array sum for the given elements. Show your solution using FIND-MAXIMUM-SUBARRAY algorithm.

$$\{-2, -1, -3, -4, -1, -2, -1, -5, -4\}$$